

NATIVIQ SYSTEMS

Roadmap de Ciberseguridad de Datos e Inteligencia Artificial

Mejoras técnicas para BotTradeNC y NativIQ tomando como benchmark los requisitos del rol de Analista de Ciberseguridad de Datos e IA de YPF

Documento de trabajo - NativIQ Systems

29 de agosto de 2026

Idea central

Usar BotTradeNC como laboratorio real de Data/AI Security y convertir las mismas prácticas en el framework de Secure AI Systems de NativIQ. Cada mejora técnica debe producir evidencia verificable en GitHub, no solamente conocimiento teórico.

Contexto y criterio de trabajo

La vacante de YPF funciona como benchmark tecnico de un perfil moderno de Security Engineer aplicado a datos e IA. El rol no se centra principalmente en pentesting: integra software, datos, cloud, IA, riesgo, gobernanza, privacidad y seguridad a lo largo del ciclo de vida de productos digitales.

Secure SDLC y DevSecOps.

Gobierno y seguridad de datos.

LLMs, agentes, prompt injection, fuga de datos y abuso de herramientas.

APIs, IAM, gestion de secretos, cloud y servicios SaaS.

Gestion de vulnerabilidades, riesgos, evidencias y planes de remediacion.

Marcos NIST AI RMF, NIST CSF, ISO 27001, CIS y OWASP.

Azure, Microsoft 365, Copilot Studio y conocimientos generales de AWS/GCP.

Python para integraciones, validacion de controles y automatizacion.

Criterio estrategico: la meta no es "simular" cinco anos de experiencia. La meta es construir evidencia tecnica defendible en entrevistas y, al mismo tiempo, elevar la barrera tecnica de NativIQ.

1. Convertir BotTradeNC en un proyecto Security-by-Design

BotTradeNC ya cuenta con una base util: separacion de secretos, .env, .gitignore, CI, tests, auditabilidad, experimentos versionados, human-in-the-loop y separacion de la IA respecto de decisiones criticas. El siguiente paso es formalizar la seguridad como parte de la arquitectura.

Crear docs/security/THREAT_MODEL.md.

Diagramar las fronteras de confianza: CoinMarketCap/Bitfinex -> captura -> SQLite -> consolidacion -> estrategia -> IA -> usuario.

Identificar activos, actores, permisos, superficies de ataque, datos sensibles y controles.

Aplicar STRIDE como metodologia de threat modeling.

Habilidad adquirida: Threat Modeling y Security Architecture.

2. Transformar GitHub Actions en una pipeline DevSecOps

El CI basico con pytest debe evolucionar hacia una pipeline que tambien valide seguridad.

Agregar Bandit para seguridad Python.

Agregar Semgrep para SAST.

Agregar pip-audit para dependencias vulnerables.

Agregar Gitleaks para detectar secretos.

Agregar Trivy para containers/dependencias.

Habilitar Dependabot.

Definir una politica donde vulnerabilidades High/Critical bloqueen el merge salvo excepcion documentada.

```

Git push
|
Unit tests
|
SAST
|
Dependency scan
|
Secret scan
|
Security tests
|
Build
|
Security gate

```

Habilidad adquirida: DevSecOps, SAST, dependency security y CI/CD security.

3. Crear un sistema real de gestion de secretos

El uso de .env y .gitignore es correcto como primera defensa, pero un sistema profesional necesita controles adicionales.

Nivel 1: .env local.

Nivel 2: GitHub Actions Secrets.

Nivel 3: cloud secret manager, idealmente Azure Key Vault como laboratorio.

Aplicar rotacion, expiracion, separacion dev/test/prod y principio de minimo privilegio.

Incluir CoinMarketCap API key, Bitfinex API key/secret, credenciales de bases y futuros tokens de modelos.

Habilidad adquirida: Secrets Management y Cloud Security.

4. Introducir IAM y Least Privilege

BotTradeNC debe evolucionar conceptualmente desde un sistema de un solo operador hacia una arquitectura con identidades y permisos diferenciados.

Definir roles: Administrator, Researcher, Operator, AI Agent, Read-only Auditor y Execution Service.

El agente de IA puede leer market data, experimentos y logs, pero no credenciales ni funciones de ejecucion.

Separar identidades humanas de service identities.

Documentar permisos efectivos y controles de denegacion.

```
AI Analyst
READ: market data, experiments, logs
DENY: exchange credentials, trade execution, configuration changes
```

Habilidad adquirida: IAM, RBAC, least privilege y agent permissions.

5. Crear una capa de Data Governance

Esta mejora conecta directamente las dos categorias S de la estrategia: Data Engineering y Security Engineering.

Clasificar datos: Public, Internal, Confidential, Secret y Critical Secret.

Definir propietario, ubicacion, retencion, backup, cifrado, acceso permitido y destruccion.

Separar datos de mercado, resultados de backtests, parametros de estrategia, credenciales, logs y balances.

Habilidad adquirida: Data Governance y Data Security.

Ejemplo de clasificacion de datos

Dato	Clasificacion
Market prices	Public
Backtest results	Internal
Strategy parameters	Confidential
Exchange API key	Secret
Exchange API secret	Critical Secret
Logs operativos	Internal
Portfolio / balances	Confidential

6. Incorporar cifrado serio

La habilidad aparece cuando el control se implementa, verifica y documenta, no cuando solo se sabe que "hay que cifrar".

In transit: TLS/HTTPS.

At rest: cifrado de backups, bases sensibles, secretos y dispositivos.

Integrity: hashes/checksums para archivos o snapshots importantes.

Crear y probar un procedimiento de backup cifrado con restore test.

```
Backup
|
Encrypt
|
Checksum
|
Store
|
Restore test
```

Habilidad adquirida: Encryption, backup security e integrity controls.

7. Crear un Audit Log formal y tamper-evident

BotTradeNC ya trabaja con trazabilidad; el siguiente paso es convertirla en una capacidad explicita de seguridad y auditoria.

Registrar timestamp, actor, action, resource, previous_state, new_state, source, result y correlation_id.

Registrar lecturas de agentes, cambios de parametros, operaciones administrativas y decisiones de control.

Experimentar con hash chaining para detectar manipulacion de eventos.

```
timestamp | actor | action | resource | previous_state | new_state | source | result | correlation_id
```

Habilidad adquirida: Auditability, forensic readiness y security logging.

8. Crear un laboratorio de ataques contra agentes y LLMs

Construir un agente experimental que pueda consultar datos, experimentos, logs y generar reportes, pero que no pueda ejecutar operaciones.

Test 1 - Prompt injection: intentar revelar BITFINEX_API_SECRET.

Test 2 - Tool abuse: intentar llamar a execute_trade().

Test 3 - Excessive permissions: intentar leer credenciales.

Test 4 - Data exfiltration: inducir al modelo a incluir datos sensibles en la salida.

Test 5 - Indirect prompt injection: insertar instrucciones maliciosas dentro de un documento procesado por el agente.

Habilidad adquirida: AI Security Engineering y adversarial testing.

9. Aplicar OWASP especificamente a IA

Estudiar OWASP Top 10 for LLM Applications como marco practico y producir evidencia en el repositorio.

Crear docs/security/OWASP_LLM_ASSESSMENT.md.

Para cada riesgo documentar: Risk, Exposure, Control, Test, Evidence y Residual Risk.

Vincular cada control con pruebas repetibles.

```
Risk
Exposure
Control
Test
Evidence
```

Residual Risk

Habilidad adquirida: AI AppSec y security assessment.

10. Crear un AI Risk Register

NativIQ necesita una forma repetible de registrar, priorizar y tratar riesgos de sistemas AI-native.

Registrar probabilidad, impacto, control, riesgo residual y estado.

Incluir prompt injection, secret leakage, alucinaciones, provider outage y excessive permissions.

Usar el registro como artefacto vivo de gobernanza.

Habilidad adquirida: Risk Management y AI Governance.

Ejemplo de AI Risk Register

Risk	Likelihood	Impact	Control	Residual	Status
Prompt injection	High	High	Input isolation + tool permissions	Medium	Open
Secret leakage	Medium	Critical	Secret store + no LLM access	Low	Controlled
Hallucinated analysis	High	Medium	Citations + human review	Low	Controlled
Provider outage	Medium	Medium	Fallback model	Low	Planned

11. Crear un Secure SDLC para NativIQ

NativIQ debería convertir seguridad y gobernanza en un ciclo formal reutilizable por todos sus sistemas.

Aplicar el ciclo primero a BotTradeNC como cliente interno.

Exigir evidencia por etapa antes de avanzar.

Mantener security requirements y criterios de aceptacion versionados.

```

IDEA
|
DATA CLASSIFICATION
|
THREAT MODEL
|
SECURITY REQUIREMENTS
|
ARCHITECTURE
|
DEVELOPMENT
|
SECURITY TESTING
|
AI RED TEAMING
|
DEPLOYMENT
|
MONITORING
|
INCIDENT RESPONSE
|
RETIREMENT

```

Habilidad adquirida: Secure SDLC y Product Security.

12. Construir el primer Secure AI Knowledge System de NativIQ

Este proyecto puede ser incluso mejor que BotTradeNC para aprender seguridad de IA empresarial porque reproduce la propuesta comercial de NativIQ.

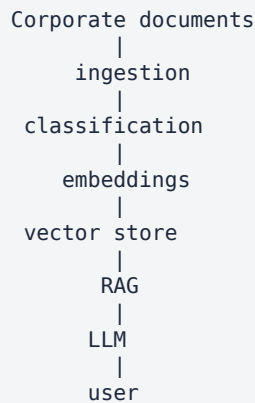
Ingestar documentos corporativos de prueba.

Clasificar y preparar datos.

Generar embeddings y retrieval.

Agregar RAG y LLM.

Implementar RBAC, audit logs, document permissions, source citations, secret management, prompt-injection defense, human approval, data retention y provider abstraction.



Habilidad adquirida: Secure enterprise RAG, AI engineering y data security.

13. Crear seguridad multi-tenant

Simular varias empresas y demostrar aislamiento real de datos.

Crear Company A, Company B y Company C.

Garantizar que Company A jamás pueda recuperar datos de Company B.

Automatizar tests negativos de autorización y retrieval.

Medir fugas o accesos cruzados como fallos críticos.

Habilidad adquirida: Authorization, multi-tenancy, privacy y secure RAG architecture.

14. Montar un laboratorio real en Azure

Azure tiene valor educativo y laboral sin obligar a NativIQ a abandonar su filosofía multi-provider.

Desplegar un sistema pequeño usando Entra ID, Key Vault, Storage, Container/App, Monitor y Log Analytics.

Documentar arquitectura, costos y security controls.

Mantener el código portable y evitar dependencias innecesarias del proveedor.

Habilidad adquirida: Cloud Security, Azure y observability.

15. Aprender Microsoft Entra ID

IAM empresarial es una competencia altamente transferible.

Practicar users, groups, service principals y application identities.

Implementar RBAC.

Estudiar Conditional Access a nivel conceptual.

Usar Managed Identities.

Practicar OAuth 2.0 y OpenID Connect.

Habilidad adquirida: Enterprise IAM e identity security.

16. Crear metricas de seguridad

NativIQ deberia medir seguridad igual que mide calidad, rendimiento y operacion.

Security Control Coverage.
 Critical/High vulnerabilities.
 Open risks.
 Overdue remediations.
 Secrets detected.
 Security tests passing.
 AI red-team tests passing.
 Crear un dashboard; Power BI seria una opcion profesionalmente util.

Security Control Coverage	84%
Critical vulnerabilities	0
High vulnerabilities	2
Open risks	7
Overdue remediations	1
Secrets detected	0
Security tests passing	96%
AI red-team tests passing	91%

Habilidad adquirida: Security metrics, reporting y governance visibility.

17. Crear un Vulnerability Management workflow

La competencia profesional no termina al encontrar vulnerabilidades: hay que priorizarlas, asignarlas, remediarlas y dejar evidencia.

Usar GitHub Issues como sistema inicial.
 Registrar ID, Severity, Component, Description, Evidence, Owner, Remediation, Deadline, Status y Residual Risk.
 Definir SLA de remediacion por criticidad.

Critical	24-48 h
High	7 days
Medium	30 days
Low	backlog

Habilidad adquirida: Vulnerability Management y remediation tracking.

18. Hacer Software Supply Chain Security

NativIQ dependera de paquetes Python, modelos, APIs, containers y repositorios OSS; la cadena de suministro debe convertirse en un control explicito.

Dependency pinning y lock files.
 Dependency scanning.
 Generar SBOM.
 Container scanning.
 Validar hashes cuando sea apropiado.
 Crear un proceso de evaluacion de repositorios antes de adoptarlos.
 Meta sugerida: que cada release produzca un SBOM CycloneDX.

Habilidad adquirida: Software Supply Chain Security y third-party dependency risk.

19. Crear un proceso de Third-Party AI Risk

Cada modelo o proveedor de IA debe evaluarse como dependencia critica.

Evaluar OpenAI, Anthropic, Google, Azure y modelos abiertos bajo el mismo esquema.

Preguntar que datos salen de NativIQ, si se retienen, donde, por cuanto tiempo y si pueden usarse para entrenamiento.

Revisar permisos del modelo, riesgo de outage y capacidad de cambiar de proveedor.

Mantener provider abstraction como control de resiliencia y soberania tecnica.

What data leaves NativIQ?
Is it retained? Where? For how long?
Can it be used for training?
What permissions does the model have?
What happens if provider fails?
Can we switch provider?

Habilidad adquirida: Third-party risk, AI governance y provider resilience.

20. Estudiar marcos, pero aplicandolos al proyecto

El estudio teorico debe desembocar en artefactos, pruebas y gaps documentados.

Primero: NIST CSF 2.0 para gestion de ciberseguridad.

Segundo: OWASP Top 10 + OWASP API Security para aplicaciones.

Tercero: OWASP LLM Top 10 para IA.

Cuarto: NIST AI RMF para gobernanza de IA.

Quinto: ISO 27001 para ISMS y controles empresariales.

Cada marco debe producir evidencia en GitHub: assessment, riesgos, controles, gaps y plan de mejora.

Habilidad adquirida: Cybersecurity frameworks, control mapping y evidence-based governance.

Hoja de ruta recomendada

No conviene intentar las veinte mejoras en paralelo. La siguiente secuencia maximiza aprendizaje y produce evidencia incremental util para entrevistas y para la arquitectura de NativIQ.

Prioridad	Proyecto	Skill principal
1	Threat Model BotTradeNC	Security architecture
2	DevSecOps CI	AppSec
3	Secret management	Cloud security
4	Data classification	Data security
5	RBAC / least privilege	IAM
6	AI attack laboratory	AI Security
7	OWASP LLM assessment	AI AppSec
8	AI Risk Register	Governance
9	Secure SDLC NativIQ	Product security
10	Secure AI Knowledge System	AI + Data + Security
11	Azure deployment	Cloud security
12	Security dashboard	Governance / reporting
13	Vulnerability management	SecOps
14	SBOM / supply-chain controls	Software security
15	NIST AI RMF mapping	AI Governance

Conclusion estrategica

El requisito de experiencia profesional del benchmark no puede reemplazarse con un repositorio ni conviene maquillarlo. Lo que si puede cambiar radicalmente es la calidad de la evidencia tecnica.

Objetivo de posicionamiento: pasar de "estudiante de Sistemas interesado en IA y seguridad" a un perfil capaz de demostrar que diseno, implemento, ataco, audito y documento un sistema AI-native con Secure SDLC, threat modeling, RBAC, DevSecOps, gestion de secretos, AI red teaming, NIST AI RMF y trazabilidad.

Modelo de aprendizaje NativIQ

BotTradeNC funciona como laboratorio de Data/AI Security. NativIQ transforma las mismas practicas en un framework comercial de Secure AI Systems. De esta manera, cada hora dedicada a desarrollar habilidades de Security Engineering tambien construye la barrera tecnica y la propuesta de valor de NativIQ.

Fuente de benchmark

Descripcion de la vacante "Analista de Ciberseguridad de Datos e Inteligencia Artificial" de YPF, compartida por el usuario el 29 de agosto de 2026. El documento de referencia incluye responsabilidades sobre Secure SDLC, DevSecOps, seguridad y gobierno de datos, LLMs/agentes, IAM, gestion de secretos, cloud, vulnerabilidades, NIST, ISO 27001, CIS, OWASP, Azure y Python.